



Artificial Intelligence BS(AI)
Shaheed Zulfiqar Ali Bhutto Institute of Science and Technology
(SZABIST)

Lab 1: Perceptron From Scratch

Objective:

To introduce perceptron, the simplest artificial neuron model, and to build hands-on understanding of how a perceptron computes a prediction, measures its error, and updates its own weights through the perceptron learning rule. By the end of this lab, students will implement a perceptron from scratch using only Python and NumPy, train it successfully on the AND gate, observe why a single perceptron fails on the XOR gate, and package the algorithm into a reusable Perceptron class.

Software and Tools:

1.6 GHz or faster processor
4 GB of RAM
20 GB of available hard disk space
Windows 7 or later
Google Colab

Colab Notebook for Lab 2:

https://colab.research.google.com/drive/1Ui7ZhaDZKuf0tiLTf4kqaJFpVA_IHOGc?usp=sharing

Background and Procedure:

Introduction

Before larger neural networks can be understood, a student must first understand their smallest building block: the perceptron. Introduced by Frank Rosenblatt in 1958, the perceptron is a simplified mathematical model of a biological neuron. It takes one or more inputs, multiplies each by a weight, adds a bias term, and passes the result through a step activation function to produce a binary output. What makes the perceptron interesting is not just this calculation, but the fact that it can learn: by comparing its prediction to the correct answer and adjusting its own weights, it gradually improves. The core learning cycle used throughout this lab is:

Predict → Calculate Error → Update Weights → Repeat

1. What Is a Perceptron?

A perceptron computes a weighted sum of its inputs plus a bias, then applies a step function to turn that sum into a 0/1 prediction:

$$z = w_1 * x_1 + w_2 * x_2 + b$$

prediction = 1 if $z \geq 0$, otherwise 0

Key concepts used throughout this lab:

- Weights (w_1 , w_2): control how much influence each input has on the output.
- Bias (b): shifts the decision boundary independently of the inputs.



Artificial Intelligence BS(AI)
Shaheed Zulfiqar Ali Bhutto Institute of Science and Technology
(SZABIST)

- Weighted sum (z): the linear combination of inputs, weights, and bias.
- Step function: converts the weighted sum into a binary prediction (0 or 1).
- Learning rate: controls how large a step is taken when correcting a mistake.

Processing pipeline: Inputs → Weighted Sum → Step Function → Prediction.

The AND Gate Dataset

The first task is to teach a perceptron the logical AND operation, which outputs 1 only when both inputs are 1:

x1	x2	Correct Output
0	0	0
0	1	0
1	0	0
1	1	1

```
▶ # Input data
x = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

# Output
y = np.array([0, 0, 0, 1])

print("Inputs:")
print(X)

print("\nCorrect outputs:")
print(y)
```

Initializing the Perceptron

Before any training happens, the perceptron knows nothing, so all weights and the bias start at zero, and a learning rate of 0.1 is chosen to control the size of each correction.

```
w1 = 0
w2 = 0
bias = 0

learning_rate = 0.1

print("w1 =", w1)
print("w2 =", w2)
print("bias =", bias)
print("learning rate =", learning_rate)
```



Artificial Intelligence BS(AI)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
(SZABIST)

The Step Activation Function

The step function is what turns a raw weighted sum into a binary decision: any value of z that is 0 or greater is treated as class 1; anything below 0 is treated as class 0.

```
def step(z):  
    if z >= 0:  
        return 1  
    else:  
        return 0  
  
print(step(5))  
print(step(-2))
```

Making a Manual Prediction

Using the initial (all-zero) weights and the input ($x_1=1$, $x_2=1$), the weighted sum works out to $z = (0)(1) + (0)(1) + 0 = 0$, which the step function classifies as 1.

```
x1 = 1  
x2 = 1  
  
z = w1*x1 + w2*x2 + bias  
  
prediction = step(z)  
  
print("x1 =", x1)  
print("x2 =", x2)  
print("Weighted sum z =", z)  
print("Prediction =", prediction)
```

Calculating the Error

The error tells the perceptron how wrong its prediction was, and in which direction:

$$\text{error} = \text{actual} - \text{prediction}$$

An error of 0 means the prediction was correct; +1 or -1 means a correction is needed.

```
actual = 1  
  
error = actual - prediction  
  
print("Actual =", actual)  
print("Prediction =", prediction)  
print("Error =", error)
```



Artificial Intelligence BS(AI)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
(SZABIST)

Updating the Weights (The Learning Rule)

When a mistake is made, every weight and the bias are nudged in the direction that would have produced the correct answer:

$$w_i = w_i + \text{learning_rate} * \text{error} * x_i$$

$$b = b + \text{learning_rate} * \text{error}$$

```
# Save the old values so we can compare them
old_w1 = w1
old_w2 = w2
old_bias = bias

# Update weights and bias
w1 = w1 + learning_rate * error * x1
w2 = w2 + learning_rate * error * x2
bias = bias + learning_rate * error

print("Old w1 =", old_w1, "-> New w1 =", w1)
print("Old w2 =", old_w2, "-> New w2 =", w2)
print("Old bias =", old_bias, "-> New bias =", bias)
```

Testing the Trained Perceptron

Once training stops, the learned weights and bias are used to re-check every example in the dataset.

```
# Reset the model
w1 = 0
w2 = 0
bias = 0

# One training example
x1 = 1
x2 = 0
actual = 0

# Prediction
z = w1*x1 + w2*x2 + bias
prediction = step(z)

# Error
error = actual - prediction

print("Weighted sum =", z)
print("Prediction =", prediction)
print("Actual =", actual)
print("Error =", error)
```



Artificial Intelligence BS(AI)
Shaheed Zulfiqar Ali Bhutto Institute of Science and Technology
(SZABIST)

```
# Update after the mistake
w1 = w1 + learning_rate * error * x1
w2 = w2 + learning_rate * error * x2
bias = bias + learning_rate * error
```

```
print("New w1 =", w1)
print("New w2 =", w2)
print("New bias =", bias)
```

```
) # Reset weights before training
w1 = 0
w2 = 0
bias = 0

epochs = 10

for epoch in range(epochs):

    errors = 0

    # Go through all training examples
    for i in range(len(X)):

        x1 = X[i][0]
        x2 = X[i][1]
        actual = y[i]

        # Step 1: weighted sum
        z = w1*x1 + w2*x2 + bias

        # Step 2: prediction
        prediction = step(z)

        # Step 3: error
        error = actual - prediction

        # Step 4: update
        w1 = w1 + learning_rate * error * x1
        w2 = w2 + learning_rate * error * x2
        bias = bias + learning_rate * error

        # Count mistakes
        if error != 0:
            errors += 1

    print("Epoch:", epoch + 1, "| Errors:", errors)

    # Stop when all examples are correct
    if errors == 0:
        break

print("\nFinal w1 =", w1)
print("Final w2 =", w2)
print("Final bias =", bias)
```



Artificial Intelligence BS(AI)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
(SZABIST)

```
print("AND predictions:")

for i in range(len(X)):

    x1 = X[i][0]
    x2 = X[i][1]

    z = w1*x1 + w2*x2 + bias
    prediction = step(z)

    print(
        "Input:", [x1, x2],
        "| z =", z,
        "| Prediction =", prediction,
        "| Actual =", y[i]
    )
```

```
AND predictions:
Input: [np.int64(0), np.int64(0)] | z = -0.20000000000000004 | Prediction = 0 | Actual = 0
Input: [np.int64(0), np.int64(1)] | z = -0.10000000000000003 | Prediction = 0 | Actual = 0
Input: [np.int64(1), np.int64(0)] | z = -2.7755575615628914e-17 | Prediction = 0 | Actual = 0
Input: [np.int64(1), np.int64(1)] | z = 0.1 | Prediction = 1 | Actual = 1
```

Visualizing the AND Data

Because the AND gate has only two inputs, the four data points can be plotted directly on a 2-D graph, with class 0 and class 1 points shown using different markers.

```
import matplotlib.pyplot as plt

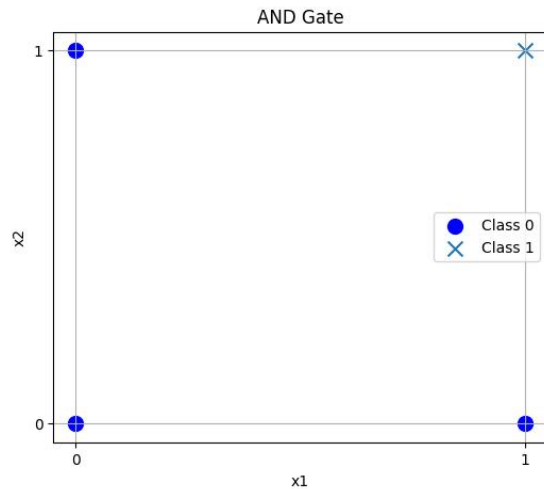
plt.figure(figsize=(6, 5))

for i in range(len(X)):
    if y[i] == 0:
        plt.scatter(X[i][0], X[i][1], marker='o', color='b', s=100, label='Class 0' if i == 0 else "")
    else:
        plt.scatter(X[i][0], X[i][1], marker='x', s=100, label='Class 1')

plt.xlabel("x1")
plt.ylabel("x2")
plt.title("AND Gate")
plt.xticks([0, 1])
plt.yticks([0, 1])
plt.grid(True)
plt.legend()
plt.show()
```



Artificial Intelligence BS(AI)
Shaheed Zulfiqar Ali Bhutto Institute of Science and Technology
(SZABIST)



The XOR Problem

Next, the exact same algorithm is applied to a harder problem: the XOR gate, which outputs 1 only when the two inputs are different.

x1	x2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Training a Perceptron on XOR

The identical training loop from Section 9 is reused, unchanged, on the XOR dataset.

```
X_xor = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y_xor = np.array([0, 1, 1, 0])

print("XOR inputs:")
print(X_xor)

print("\nXOR outputs:")
print(y_xor)
```

Why XOR Fails: Linear Separability

A single perceptron can only draw one straight decision line ($w_1 \cdot x_1 + w_2 \cdot x_2 + b = 0$) through the input space. For AND, a single line can place the one “1” point on one side and the three “0” points on the other. For XOR, the two class-1 points and two class-0 points are arranged diagonally, so no single straight line can separate them. This is exactly what “not linearly separable” means, and it is a fundamental limitation



Artificial Intelligence BS(AI)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
(SZABIST)

of the model itself, not a bug in the training code — no amount of additional training fixes it. This limitation is precisely what motivates multilayer neural networks, introduced later in this course.

```
# Reset weights
w1 = 0
w2 = 0
bias = 0

epochs = 20

for epoch in range(epochs):

    errors = 0

    for i in range(len(X_xor)):

        x1 = X_xor[i][0]
        x2 = X_xor[i][1]
        actual = y_xor[i]

        z = w1*x1 + w2*x2 + bias
        prediction = step(z)

        error = actual - prediction

        w1 = w1 + learning_rate * error * x1
        w2 = w2 + learning_rate * error * x2
        bias = bias + learning_rate * error

        if error != 0:
            errors += 1

    print("Epoch:", epoch + 1, "| Errors:", errors)

print("\nFinal w1 =", w1)
print("Final w2 =", w2)
print("Final bias =", bias)

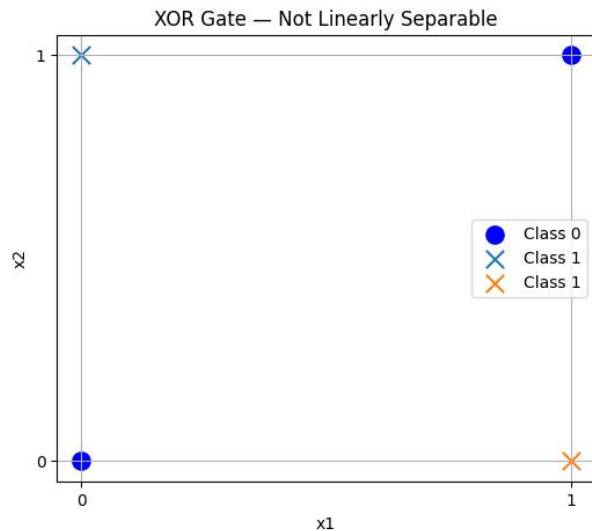
plt.figure(figsize=(6, 5))

for i in range(len(X_xor)):
    if y_xor[i] == 0:
        plt.scatter(
            X_xor[i][0], X_xor[i][1],
            marker='o', color='b', s=120,
            label='Class 0' if i == 0 else ""
        )
    else:
        plt.scatter(
            X_xor[i][0], X_xor[i][1],
            marker='x', s=120,
            label='Class 1'
        )

plt.xlabel("x1")
plt.ylabel("x2")
plt.title("XOR Gate - Not Linearly Separable")
plt.xticks([0, 1])
plt.yticks([0, 1])
plt.grid(True)
plt.legend()
plt.show()
```




Artificial Intelligence BS(AI)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
(SZABIST)



```
class Perceptron:

    def __init__(self, learning_rate=0.1):
        self.w1 = 0
        self.w2 = 0
        self.bias = 0
        self.learning_rate = learning_rate

    def step(self, z):
        if z >= 0:
            return 1
        else:
            return 0

    def predict_one(self, x1, x2):
        z = self.w1*x1 + self.w2*x2 + self.bias
        return self.step(z)

    def fit(self, X, y, epochs=10):

        for epoch in range(epochs):

            errors = 0

            for i in range(len(X)):

                x1 = X[i][0]
                x2 = X[i][1]
                actual = y[i]

                prediction = self.predict_one(x1, x2)

                error = actual - prediction

                self.w1 = self.w1 + self.learning_rate * error * x1
                self.w2 = self.w2 + self.learning_rate * error * x2
                self.bias = self.bias + self.learning_rate * error
```



Artificial Intelligence BS(AI)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
(SZABIST)

```
if error != 0:
    errors += 1

print("Epoch:", epoch + 1, "| Errors:", errors)

if errors == 0:
    break
```

```
p = Perceptron(learning_rate=0.1)

p.fit(X, y)

print("\nLearned values:")
print("w1 =", p.w1)
print("w2 =", p.w2)
print("bias =", p.bias)
```

```
Epoch: 1 | Errors: 2
Epoch: 2 | Errors: 3
Epoch: 3 | Errors: 3
Epoch: 4 | Errors: 0
```

```
Learned values:
w1 = 0.2
w2 = 0.1
bias = -0.20000000000000004
```

```
print("Final predictions:")

for i in range(len(X)):

    x1 = X[i][0]
    x2 = X[i][1]

    prediction = p.predict_one(x1, x2)

    print(
        "Input:", [x1, x2],
        "| Prediction:", prediction,
        "| Actual:", y[i]
    )
```

```
Final predictions:
Input: [np.int64(0), np.int64(0)] | Prediction: 0 | Actual: 0
Input: [np.int64(0), np.int64(1)] | Prediction: 0 | Actual: 0
Input: [np.int64(1), np.int64(0)] | Prediction: 0 | Actual: 0
Input: [np.int64(1), np.int64(1)] | Prediction: 1 | Actual: 1
```



Artificial Intelligence BS(AI)
Shaheed Zulfiqar Ali Bhutto Institute of Science and Technology
(SZABIST)

Lab Tasks:

1. Take the pre-set OR gate dataset and train your perceptron model on it. Observe the training process to determine whether the model successfully learns the pattern and finishes training without mistakes, and count the exact number of cycles (epochs) it takes to reach that point.
2. Using the pre-defined Perceptron class, write a brief script that initializes a fresh perceptron object, runs it through the training process using the OR gate data, and then displays the final adjusted weights, the bias value, and the network's output predictions for all four possible combinations of inputs.

Submission:

- Submit the Colab notebook with all cells executed and outputs visible.
- Ensure the notebook is well-commented and demonstrates your understanding of each operation.